

AURA

Autonomous User Response Agent

Software Development Lifecycle (SDLC) Documentation

Document title	AURA — SDLC Documentation Pack
Version	1.0 (Baseline)
Status	Draft for review
Date	22 June 2026
Author / Owner	Anurag Kumar Singh
Classification	Confidential
Applicable standards	IEEE 830, ISO/IEC 25010, ISO/IEC 27001, OWASP ASVS, NIST AI RMF 1.0

This pack documents AURA across all six lifecycle phases — Planning, Analysis, Design, Implementation, Testing & Integration, and Maintenance — at the level expected of a production engineering organisation.

Document Control

Ver.	Date	Author	Summary of change
0.1	18 Jun 2026	A. K. Singh	Initial skeleton and scope
0.5	20 Jun 2026	A. K. Singh	Architecture and requirements drafted
1.0	22 Jun 2026	A. K. Singh	Baseline approved for review

Distribution: project owner, contributing engineers, security reviewer. Change requests are tracked in the issue tracker and reflected in this table on each baseline.

Table of Contents

Project Charter & Vision / Scope

5

1.1 Purpose

5

1.2 Vision

5

1.3 Problem statement

5

1.4 Objectives and success metrics

5

1.5 Scope

5

1.6 Stakeholders

6

1.7 Assumptions and constraints

6

1.8 High-level roadmap

6

1.9 Risk register

6

1.10 Ethics, safety and responsible AI

7

Software Requirements Specification (SRS)

8

2.1 Purpose and scope

8

2.2 User personas

8

2.3 Functional requirements

8

2.4 Non-functional requirements

8

2.5 External interfaces

9

2.6 Data requirements

9

2.7 Constraints, assumptions, dependencies

9

2.8 Acceptance criteria

9

Software Design Document (SDD)

10

3.1 Architectural goals

10

3.2 Design principles — SOLID mapping

10

3.3 Layered architecture

10

3.4 Key interfaces

10

3.5 Security architecture and threat model

11

3.6 Model routing strategy

11

3.7 Primary sequence — voice command to action

11

3.8 Technology stack

11

3.9 Architectural decision records (selected)

12

Implementation & Coding Guide

13

4.1 Repository structure	13
4.2 Coding standards	13
4.3 Module catalogue	13
4.4 Build order (MVP to full)	13
4.5 Version control and workflow	13
4.6 Definition of Done	13
Test & Integration Plan	15
5.1 Objectives	15
5.2 Test levels	15
5.3 Agent reliability evaluation	15
5.4 Security testing	15
5.5 Sample test cases	15
5.6 Continuous integration and exit criteria	16
Operations, Release & Maintenance Plan	17
6.1 Packaging and distribution	17
6.2 Release management	17
6.3 Monitoring and observability	17
6.4 Incident response and security patching	17
6.5 Support, data retention and privacy	17
6.6 Roadmap and future enhancements	17
6.7 Deprecation and end of life	17
Appendix A - Glossary	18
Appendix B - Requirements traceability (sample)	18
Appendix C - Reference standards	18

DOCUMENT 1 | PHASE 1 — PLANNING

Project Charter & Vision / Scope

1.1 Purpose

This charter authorises the AURA project, fixes its scope, and aligns stakeholders on what success means before engineering begins. It is the single source of truth for why the project exists and where its boundaries lie.

1.2 Vision

AURA is a local-first, voice- and face-driven desktop agent — a practical assistant that understands a spoken instruction (including Hindi and Hinglish), reads what is on screen, plans the steps required, and operates the machine through a controlled set of tools, then reports back. The long-term ambition is broad, general task assistance. To stay credible and shippable, that ambition is treated as a guiding direction, not a delivered capability: AURA targets reliable automation of well-defined desktop and knowledge tasks, expanding outward one verified capability at a time.

1.3 Problem statement

Existing assistants are either cloud-locked (privacy and offline concerns), shallow (single-turn questions, no real action), or brittle in demos because they ignore failure handling and safety. There is no open, local-first agent that combines multimodal understanding, genuine OS-level action, and a disciplined safety model suitable for everyday trust.

1.4 Objectives and success metrics

#	Objective	Measurable success metric
O1	Reliable task execution	Task success rate $\geq 90\%$ on the defined task suite
O2	Safety by default	0 unconfirmed sensitive/destructive actions in audit log
O3	Privacy / local-first	Core loop runnable fully offline (local model + local I/O)
O4	Responsiveness	Median voice-to-first-action latency ≤ 3.5 s (hybrid mode)
O5	Maintainability	New tool or model provider added without core edits; CI green
O6	Distributable	One-click desktop installer for Windows/macOS/Linux

1.5 Scope

In scope

- Voice input (multilingual), text-to-speech output, and wake-word activation.
- Face-based operator verification as a gate for sensitive actions.
- Agentic planner with tool-calling over a registry of OS, email, web and screen-reading tools.
- Multi-model routing: Gemini Flash and Pro for cloud reasoning, a local model for offline use.
- A central security policy: risk tiers, identity gate, explicit confirmation, audit logging.
- Desktop packaging and a licensing hook for paid distribution.

Out of scope (this baseline)

- Artificial General Intelligence or fully autonomous, unsupervised operation. AURA always keeps a human confirmation step on irreversible actions; “end to end” means the safe majority is automated, not that oversight is removed.
- Mobile and browser-extension clients (candidate for a later release).
- Multi-user / enterprise identity federation.

1.6 Stakeholders

Stakeholder	Role	Primary interest
Project owner	Vision, priorities, sign-off	Impact, viability, timeline
Engineering	Build, test, maintain	Clear requirements, clean architecture
Security reviewer	Risk and safety gate	No unsafe action paths; data protection
End users	Daily operation	Reliability, privacy, ease of use
Prospective buyers	Purchase / adoption	Trust, security posture, support

1.7 Assumptions and constraints

- Assumption: the operator owns the machine and consents to automation on it.
- Assumption: a cloud API key is available for hybrid mode; offline mode needs a local model installed.
- Constraint: cross-application control via OS automation is inherently brittle; real APIs are preferred.
- Constraint: multimodal (screen) calls cost more and are slower, so they are used only when needed.

1.8 High-level roadmap

Milestone	Outcome	Indicative window
M1 Core loop	Text-mode planner + 3 tools, tests green	Weeks 1
M2 Voice	ASR + TTS + wake word	Weeks 2
M3 Identity	Face verification gating sensitive tools	Weeks 3
M4 Reach	Browser + email + screen-reading tools	Weeks 4-5
M5 Hardening	Audit log, eval harness, security pass	Weeks 6
M6 Release	Signed installers + licensing	Weeks 7-8

1.9 Risk register

ID	Risk	L	I	Mitigation
R1	LLM takes an unintended harmful action	M	H	Risk tiers + identity gate + mandatory confirmation; allow-listed shell
R2	OS automation fails / UI changes	H	M	Prefer real APIs; structured ToolResult; planner re-plans on failure
R3	Secret/API key leakage	L	H	Env-only secrets, log redaction filter, never persisted in plaintext
R4	Cloud outage or quota exhaustion	M	M	Automatic fallback to local model; degrade, do not crash
R5	Latency hurts usability	M	M	Flash for routine planning; local ASR; vision only on demand
R6	Scope creep toward “AGI”	M	M	Charter fixes scope; capabilities added only with passing evals

L = Likelihood, I = Impact (L/M/H).

1.10 Ethics, safety and responsible AI

AURA can act on a user's behalf, so responsibility is a first-class requirement, not an afterthought. The project adopts the NIST AI Risk Management Framework functions (Govern, Map, Measure, Manage): every action is classified by risk, irreversible actions require explicit human consent, all actions are logged for accountability, and operator identity is verified before anything sensitive runs. Data stays on the device by default.

DOCUMENT 2 | PHASE 2 — ANALYSIS

Software Requirements Specification (SRS)

2.1 Purpose and scope

This SRS specifies the functional and non-functional requirements for AURA, following the structure of IEEE 830. It is the contract between intent and implementation: every requirement is uniquely identified so it can be traced into design, code and tests.

2.2 User personas

- **The owner-operator** — a technical individual who wants a private, powerful assistant on their own machine and is comfortable granting it real control under supervision.
- **The non-technical buyer** — wants a simple, trustworthy installable app that saves time on repetitive desktop and communication tasks.

2.3 Functional requirements

ID	Requirement	Priority
FR-1	Activate on a configurable wake word.	Should
FR-2	Transcribe spoken input, including Hindi and Hinglish, to text.	Must
FR-3	Verify operator identity via face match before any non-safe action.	Must
FR-4	Plan a sequence of tool calls from a natural-language instruction.	Must
FR-5	Execute tools: open application, send email, run allow-listed command, web action.	Must
FR-6	Capture and interpret on-screen content on request (screen reading).	Should
FR-7	Classify each action as safe, sensitive or destructive.	Must
FR-8	Require explicit confirmation for sensitive and destructive actions.	Must
FR-9	Re-plan automatically when a tool reports failure.	Must
FR-10	Reply by speech in the same language the user used.	Should
FR-11	Persist an audit log of every request, plan, action and result.	Must
FR-12	Operate fully offline using a local model when configured.	Should

2.4 Non-functional requirements

Quality attributes are organised per ISO/IEC 25010.

ID	Attribute	Requirement
NFR-1	Security	Operator verification, action confirmation, secret redaction, allow-listed shell.
NFR-2	Privacy	Default to on-device processing; no user data sent to cloud in local mode.
NFR-3	Performance	Median voice-to-first-action ≤ 3.5 s in hybrid mode.
NFR-4	Reliability	Graceful degradation: cloud failure falls back to local model.
NFR-5	Scalability	New tools/providers added via registration without core changes.
NFR-6	Maintainability	SOLID layering; unit-testable core with the LLM faked.

ID	Attribute	Requirement
NFR-7	Usability	Confirmations are clear and state exactly what will happen.
NFR-8	Portability	Runs on Windows, macOS and Linux from one codebase.

2.5 External interfaces

- **User interface:** microphone, speaker, webcam, plus a console/text fallback for development.
- **Model interface:** Gemini (Flash, Pro) and a local model, accessed through one provider abstraction.
- **System interfaces:** OS process control, email API, browser automation, screen capture.

2.6 Data requirements

- Configuration and secrets from environment/.env; secrets never logged or committed.
- Enrolled face embedding stored locally and never transmitted.
- Conversation memory (recent turns) and a persistent JSONL audit log under the user's data directory.

2.7 Constraints, assumptions, dependencies

- Depends on availability of a cloud key (hybrid) or a locally installed model (offline).
- Assumes single trusted operator per installation in this baseline.
- Constrained by the reliability of third-party OS automation libraries.

2.8 Acceptance criteria

The analysis phase is complete when every functional requirement maps to at least one design element and one planned test (see the traceability matrix in Appendix B), and all Must-priority requirements are agreed by the owner and security reviewer.

DOCUMENT 3 | PHASE 3 — DESIGN

Software Design Document (SDD)

3.1 Architectural goals

- Every external dependency (speech, vision, model, OS) sits behind a small interface.
- The orchestrator depends only on interfaces; concretes are injected by factories.
- Safety decisions live in exactly one auditable place.
- Local and cloud are a configuration switch, never a rewrite.

3.2 Design principles — SOLID mapping

Principle	How it is applied in AURA
Single responsibility	Each tool, provider and policy does exactly one thing.
Open / closed	New tools and model providers are registered, not wired into core logic.
Liskov substitution	Any Listener, Speaker or Tool is interchangeable with another of its type.
Interface segregation	Interfaces are tiny: listen, speak, verify, complete, run.
Dependency inversion	Core depends on abstractions; factories supply the implementations.

3.3 Layered architecture

AURA is organised as a pipeline of swappable layers. A request flows: wake detection, then speech-to-text, into the planner (the agentic brain), which selects tools from a registry; each tool passes through the security policy before it runs; results feed back to the planner for re-planning; the final response is spoken out. Memory and the audit log run alongside the loop.

Layer	Folder	Interface	Representative implementations
Wake word	src/io/wake	WakeDetector	AlwaysOn, openWakeWord
Speech-in	src/io/asr	Listener	Console, faster-whisper, Deepgram
Speech-out	src/io/tts	Speaker	Console, Piper, ElevenLabs
Identity	src/auth	Authenticator	Trusting (dev), InsightFace
Reasoning	src/llm	LLMProvider	litellm: Gemini Flash/Pro, local Ollama
Planner	src/agent	(core loop)	Plan-act-observe-replan
Tools	src/tools	Tool + Registry	open_app, send_email, run_command, read_screen
Security	src/security	ConfirmationPolicy	Risk tiers + identity + confirm
Memory	src/memory	ConversationMemory	In-memory, SQLite (planned)
Composition	src/factories	(wiring)	Build objects from Settings

3.4 Key interfaces

- **Listener.listen()** -> **Utterance** — capture one spoken request.
- **Speaker.speak(text)** — voice the response.

- **Authenticator.verify()** -> **bool** — confirm the enrolled operator is present.
- **LLMProvider.complete(messages, tools)** — planning with tool-calling, model-agnostic.
- **Tool.run(**args)** -> **ToolResult** — perform one action and return a uniform result.

3.5 Security architecture and threat model

Threats are reasoned about with a STRIDE-style lens and countered with defence in depth:

Threat	Example	Control
Spoofing	Someone else issues commands	Face verification before non-safe actions
Tampering	Malicious instruction in screen text	Untrusted content cannot trigger auto-execution; confirm required
Repudiation	"I never asked for that"	Append-only JSONL audit log of every action
Info disclosure	API key leaks to logs	Env-only secrets + log redaction filter
Elevation	LLM runs arbitrary shell	Allow-listed commands + destructive tier confirmation

The confirmation policy is the heart of the model: safe actions auto-run; sensitive and destructive actions require both a verified operator and an explicit yes. This single gate is small, central and testable on purpose.

3.6 Model routing strategy

One provider abstraction routes to three tiers: Gemini Flash for routine planning (cheap and fast), Gemini Pro for hard reasoning, and a local model as the offline fallback. On any cloud error the provider degrades to local rather than failing. The rest of the system is unaware which model answered.

3.7 Primary sequence — voice command to action

- Wake word detected; microphone opens.
- Speech transcribed to text (language auto-detected).
- Planner asks the model for the next tool call given the request and tool specs.
- Security policy checks identity and risk; confirms if needed.
- Tool executes and returns a structured result; planner observes and continues or finishes.
- Response is spoken; the full trace is written to the audit log.

3.8 Technology stack

Concern	Choice	Rationale
Language	Python 3.10+	Ecosystem for AI, audio, vision, OS control
Model gateway	litellm	One API for Gemini and local models
Speech-in	faster-whisper	Accurate, local, multilingual
Speech-out	Piper / cloud TTS	Local-first, optional higher-quality cloud
Vision / face	InsightFace, OpenCV	On-device embeddings and capture
Web/OS control	Playwright, native automation	Reliable, API-first where possible
Packaging	PyInstaller	Single-file desktop installers
Quality	pytest, ruff	Fast tests and consistent linting

3.9 Architectural decision records (selected)

ADR	Decision	Why
ADR-1	Interfaces + factory injection everywhere	Swap local/cloud without touching core; testability
ADR-2	Single confirmation policy object	One auditable place for the dangerous decision
ADR-3	litellm as the model gateway	Avoid vendor lock-in; offline fallback
ADR-4	Tools return data, not exceptions	Planner can recover instead of crashing
ADR-5	Console I/O as first-class dev mode	Whole loop testable with no hardware

DOCUMENT 4 | PHASE 4 — IMPLEMENTATION

Implementation & Coding Guide

4.1 Repository structure

The project uses a `src/` layout. Core packages: `core` (`config`, `logging`, `shared types`), `io` (`asr/tts/wake`), `auth`, `llm`, `agent`, `tools`, `security`, `memory`, `factories`, plus the `orchestrator` and `entry point`. Supporting folders: `tests`, `scripts`, `config` and `assets`.

4.2 Coding standards

- Follow PEP 8; enforce style and import order with `ruff` in CI.
- Type-hint public functions; prefer small, single-purpose modules.
- Docstrings explain *why*, not just what — especially for safety-relevant code.
- No secrets in code or logs; read them from the environment only.
- Never weaken or bypass the confirmation policy to make a feature easier.

4.3 Module catalogue

Module	Responsibility
<code>core/config.py</code>	Immutable, validated settings injected everywhere
<code>core/logging.py</code>	Single logger with secret-redaction filter
<code>llm/litellm_provider.py</code>	Tiered model routing with offline fallback
<code>agent/planner.py</code>	Plan-act-observe-replan loop with safety gates
<code>tools/registry.py</code>	Open/closed registry of available tools
<code>security/policy.py</code>	Risk + identity + confirmation decision
<code>factories/component_factory.py</code>	Builds concrete objects from settings
<code>orchestrator.py</code>	Wires layers and runs the top-level loop

4.4 Build order (MVP to full)

- Stage 1 — Core loop in console mode: planner plus three tools, tests green.
- Stage 2 — Voice: wire ASR, TTS and wake word behind their interfaces.
- Stage 3 — Identity: implement face verification gating sensitive tools.
- Stage 4 — Reach: add browser, email and screen-reading tools.
- Stage 5 — Hardening: audit log, eval harness, security review.

4.5 Version control and workflow

- Trunk-based development with short-lived feature branches and pull requests.
- Every PR must pass tests and lint before merge; security-relevant changes need review.
- Conventional commit messages; tag releases with semantic versions.

4.6 Definition of Done

- Code merged with passing tests and lint; new logic has tests.
- Requirement IDs referenced; docs updated where behaviour changed.

- No new secret in code or logs; safety gates intact.

DOCUMENT 5 | PHASE 5 — TESTING & INTEGRATION

Test & Integration Plan

5.1 Objectives

Testing must prove two things beyond ordinary correctness: that AURA acts reliably under real-world failure, and that it never performs an unsafe action without authorisation and consent. Reliability is measured, not assumed.

5.2 Test levels

Level	Focus	Approach
Unit	Tools, policy, config, routing	pytest with the LLM faked; no network
Integration	Planner + registry + policy together	Scripted model responses drive the loop
System	End-to-end voice/face/action	Manual and recorded scenario runs
Acceptance	Requirements satisfied	Trace each Must requirement to a passing test

5.3 Agent reliability evaluation

Because an LLM drives behaviour, AURA includes an evaluation harness distinct from unit tests. A fixed task suite is run repeatedly; each run records the request, the plan, the actions and the outcome. From the audit log the team computes task success rate, recovery rate after a tool failure, unnecessary-action rate, and grounding accuracy for screen-reading. A prompt or model change that lowers these scores is a regression, even if every unit test still passes.

Metric	Definition	Target
Task success rate	Tasks completed correctly end to end	$\geq 90\%$
Recovery rate	Failed tool calls the planner recovers from	$\geq 80\%$
Safety violations	Unconfirmed sensitive/destructive actions	0
Grounding accuracy	Screen readings that match ground truth	$\geq 90\%$

5.4 Security testing

- Verify sensitive and destructive actions are blocked without confirmation.
- Verify non-safe actions are blocked when identity is not verified.
- Verify shell commands outside the allow-list are rejected.
- Verify the log redaction filter masks key-like strings.
- Attempt prompt-injection via screen content; confirm it cannot auto-execute.

5.5 Sample test cases

ID	Scenario	Expected result
TC-1	Open an application (safe)	Runs without confirmation; success logged
TC-2	Send email (sensitive)	Blocked until explicit confirmation
TC-3	Shell command not in allow-list	Rejected with clear reason

ID	Scenario	Expected result
TC-4	Tool returns failure	Planner re-plans, does not crash
TC-5	Identity not verified	Non-safe action refused
TC-6	Hindi instruction	Understood; reply in same language

5.6 Continuous integration and exit criteria

- CI runs lint and the full unit/integration suite on every push.
- Merge is blocked on any failure; coverage of core logic is tracked.
- Release exit criteria: all Must requirements traced to passing tests, zero safety violations, evaluation metrics at or above target.

DOCUMENT 6 | PHASE 6 — MAINTENANCE

Operations, Release & Maintenance Plan

6.1 Packaging and distribution

- Build per-OS installers with PyInstaller; bundle or first-run-download local models.
- Code-sign binaries so Windows SmartScreen and macOS Gatekeeper do not block them.
- Distribute via the project site and signed releases; gate paid features with a license key checked at startup.

6.2 Release management

Releases follow semantic versioning (MAJOR.MINOR.PATCH). Each release ships notes, the requirement and ADR deltas, and a short security summary. A new capability ships only after its evaluation metrics meet target.

6.3 Monitoring and observability

- The local audit log captures every request, plan, action, result, latency and model used.
- Operators can review or export the log; it doubles as the evaluation dataset.
- Optional, opt-in, anonymised telemetry for aggregate reliability — never action content.

6.4 Incident response and security patching

Step	Action
Detect	User report or audit-log anomaly
Contain	Disable the affected tool via configuration
Fix	Patch, add a regression test, re-run the security suite
Release	Ship a patch version with notes
Review	Post-incident note; update the risk register

6.5 Support, data retention and privacy

- Documentation and an issue tracker for support; defined response targets for paid tiers.
- Audit log and face embedding stay on the device; the user can clear them at any time.
- No user content leaves the machine in local mode; cloud mode is disclosed and consented.

6.6 Roadmap and future enhancements

- Vision-guided UI actions (see-then-click) under the destructive tier with confirmation.
- Richer memory (persistent, searchable) and per-user preference profiles.
- Additional verticals: calendar, documents, messaging, developer workflows.
- Mobile companion and optional encrypted multi-device sync.

6.7 Deprecation and end of life

Deprecated tools or model providers are announced one minor version ahead, kept working through a transition window, then removed in a major version with migration notes.

APPENDICES

Glossary, Traceability & References

Appendix A - Glossary

Term	Meaning
Agentic loop	Plan, act, observe, re-plan cycle the planner runs
Risk tier	Safe / sensitive / destructive classification of an action
Confirmation policy	Central rule deciding identity and consent before an action
Tool	A single capability the agent can invoke (open app, send email, etc.)
Provider	An interchangeable implementation behind an interface
Audit log	Append-only record of every request, plan, action and result

Appendix B - Requirements traceability (sample)

Requirement	Design element	Test
FR-5 Execute tools	Tool + ToolRegistry (3.4)	TC-1, TC-2
FR-7/8 Risk + confirm	ConfirmationPolicy (3.5)	TC-2, TC-3, TC-5
FR-9 Re-plan on failure	Planner loop (3.7)	TC-4
FR-2/10 Multilingual	ASR + planner prompt (3.6)	TC-6
NFR-1 Security	Threat model (3.5)	5.4 security suite

Appendix C - Reference standards

- IEEE 830 — Recommended Practice for Software Requirements Specifications.
- ISO/IEC 25010 — Systems and software quality models.
- ISO/IEC 27001 — Information security management.
- OWASP ASVS — Application Security Verification Standard.
- NIST AI Risk Management Framework 1.0 — Govern, Map, Measure, Manage.

End of document. This baseline is maintained under version control; the latest approved version supersedes all earlier copies.